

DSA210 Spring 2026 Term Project:

A Data Driven Exploration of Pokémon

Motivation

Pokémon as a franchise has been a part of my life since childhood. When I was younger, I didn't properly understand how the games systems worked or why certain game design decisions were made. As I grew older, I found myself pondering about the game design decisions behind not only Pokémon but anything I play. I also found myself interested in the effectiveness of computer algorithms in learning the same way players do after investing time in a game. Would a computer algorithm value the same qualities or make the same decisions as a good player? Is it possible to algorithmically learn the most effective tactic available (META) of a game the same way a player base does? This project gave me the opportunity to explore these questions through the application of data science practices on past 30 years of Pokémon game data.

Data source

All raw data for this project is sourced from two APIs, PokéAPI and Smogon API. PokéAPI is an open-source RESTful API project developed to provide a central source for all online information on Pokémon video games. For this project, basic information about each Pokémon is acquired such as their base stats, types, moves, abilities and generations. This results in 6 JSONs retrieved as raw data sources (abilities_raw, generations_raw, move_damage_classes_raw, pokemon_raw, species_raw and types_raw). These JSONs contain a ton of information both relevant and irrelevant thus they were reduced down to relevant information during the data processing phase, refer to data_processing.ipynb for more details.

To explain what Smogon API is I must first explain what Smogon is. Smogon is an unofficial community of competitive Pokémon players that have well defined and regularly updated rulesets for how competitive Pokémon should be played. This community also built and maintains a web-based Pokémon battle simulator called Pokémon Showdown, this is where their rulesets (formats) are played. Usage statistics from each format is collected monthly and is referenced by a community appointed council to decide categorize Pokémon into different tiers that vaguely indicate their power-level. These tiers dictate which Pokémon are allowed in which format. Smogon API is an open-

source project that is part of a large collection of open-source projects. Smogon API provides various endpoints that provide information such as the detailed usage statistics of each playable Smogon format. In this project this API is used to fetch usage statistics of each format in order to use to calculate the tiers of each played Pokémon to later use as labels for the machine learning (ML) algorithms. The fetched JSON for each format corresponds to the most recent monthly usage statistics that have exceeded a minimum number of required battles.

Data collection from PokéAPI was automated using a script which can be found in the project repository as `raw_data_collection_from_API.ipynb`. For Smogon API automation was unnecessary as such data was fetched manually because it was just 8 JSONs each corresponding to a tier (there is 1 exception where 2 JSONs correspond to a single tier).

Data analysis

Data Processing: PokéAPI

This phase of the project mostly consisted of removing irrelevant fields from each fetch JSON, some examples of these fields are descriptions, names in different languages, encounter areas. For more detailed information refer to `data_processing.ipynb`. Other tasks in this phase included removal of non-canon content such as moves and abilities from the spin-off game Pokémon XD: Gale of Darkness and 'special' Pokémon that appear in a single game as a unique one time thing such as cosplay Pikachu and its various forms, again for exact details refer to the jupyter notebook. The final task in this phase was to map all Pokémon to their correct generation, while the fetched JSONs provided generation information there were special cases where this information was wrong. For example, all Mega Pokémon have been released either in generation 6 or generation 9 but according to the JSONs the generation of the non-mega form was listed. This is because PokéAPI classifies all different forms of a Pokémon into a single species and the generation information is only provided for species.

Data Processing: Smogon API

The data processing for data obtained from Smogon API was performed during the ML phase of the project as Smogon data was used primarily to answer the question: "Is it possible to teach an ML algorithm what makes a Pokémon competitively viable?". The processing of Smogon data involved solving two main problems; determining which tier a Pokémon belongs to and dealing with the naming convention differences between the two aggregated data sources. To determine tiers,

Smogon's rules were followed, for example here is the webpage for OU: <https://www.smogon.com/dex/sv/formats/ou/>. According to Smogon for usage-based tiers (all except for Ubers) a Pokémon needs to have at least ~4.52% weighted usage within the format corresponding to a tier to be considered a member of that tier. This was easy since the JSON for each format provides the weighted usage of each Pokémon played in the format. The not so easy part is due to how Ubers functions as a tier, Ubers is the highest power-level format where anything (except for very few) is allowed to be played and there is no usage-based membership system. The way membership in Ubers was determined in this project was to remove Pokémon determined to be in lower tiers through usage filtering performed in the other tiers, banlist Pokémon also were removed and finally manual cleanup was performed on the remaining list. The banlist for a tier is a list of Pokémon that are not allowed in that tier that also don't qualify to be a member of the tier above due to the usage-based membership rule. The second problem to solve was name mismatches such as one source using ogerpon-wellspring while the other uses ogerpon-wellspring-mask to refer to the same Pokémon. This problem was handled manually by going through each mismatch one-by-one. The details of Smogon API data processing can be found in the `data_prep_and_feature_engineering` notebook.

Exploratory Data Analysis (EDA)

In EDA I wanted to answer four questions: what is the most common move damage class (status, physical and special), what are the most common abilities, do each generation follow a similar type distribution and are Pokémon from later generations more likely to be competitively dominant. The first two questions are not very interesting I admit, but their answers were still surprising. I had expected status moves to be the most common, but it was physical instead, in hindsight even Pokémon who are special attackers tend to have access to a bunch of weak physical moves they do not use. On the abilities side of things, I did not expect intimidate, levitate and sheer force to be so common considering how useful they are.

Type distributions of each generation were an interesting topic to explore because historically some types were rare such as dragon and ghost. As new generations released it started feeling like these types, especially dragon lost its previous mystery by becoming a very common type. Some types like flying and water felt common, this made intuitive sense since all bodies of water in game are populated by water types. Visualizing the type distributions of each generation allowed me to see if my observations and intuitions were backed by data. Water and flying were indeed very common

across generations, and dragon had become a common type after staying relatively rare for the first four generations. I also wanted to see the impact of various forms on the type distributions of the generation they were released in, for example the impact of mega evolutions on the distribution of generation 6. Most mega evolutions were given to Pokémon that were already popular and some have gained dragon as a secondary typing (such as Mega Charizard X and Mega Ampharos), I had a hunch that the number of dragon types released in generation 6 was inflated due to mega evolutions. My suspicions were proved correct as of the 20 dragon types in generation 6, 9 of them were mega evolutions. All discussed distributions were visualized as horizontal bar plots where categories were separated by color and sorted by frequency.

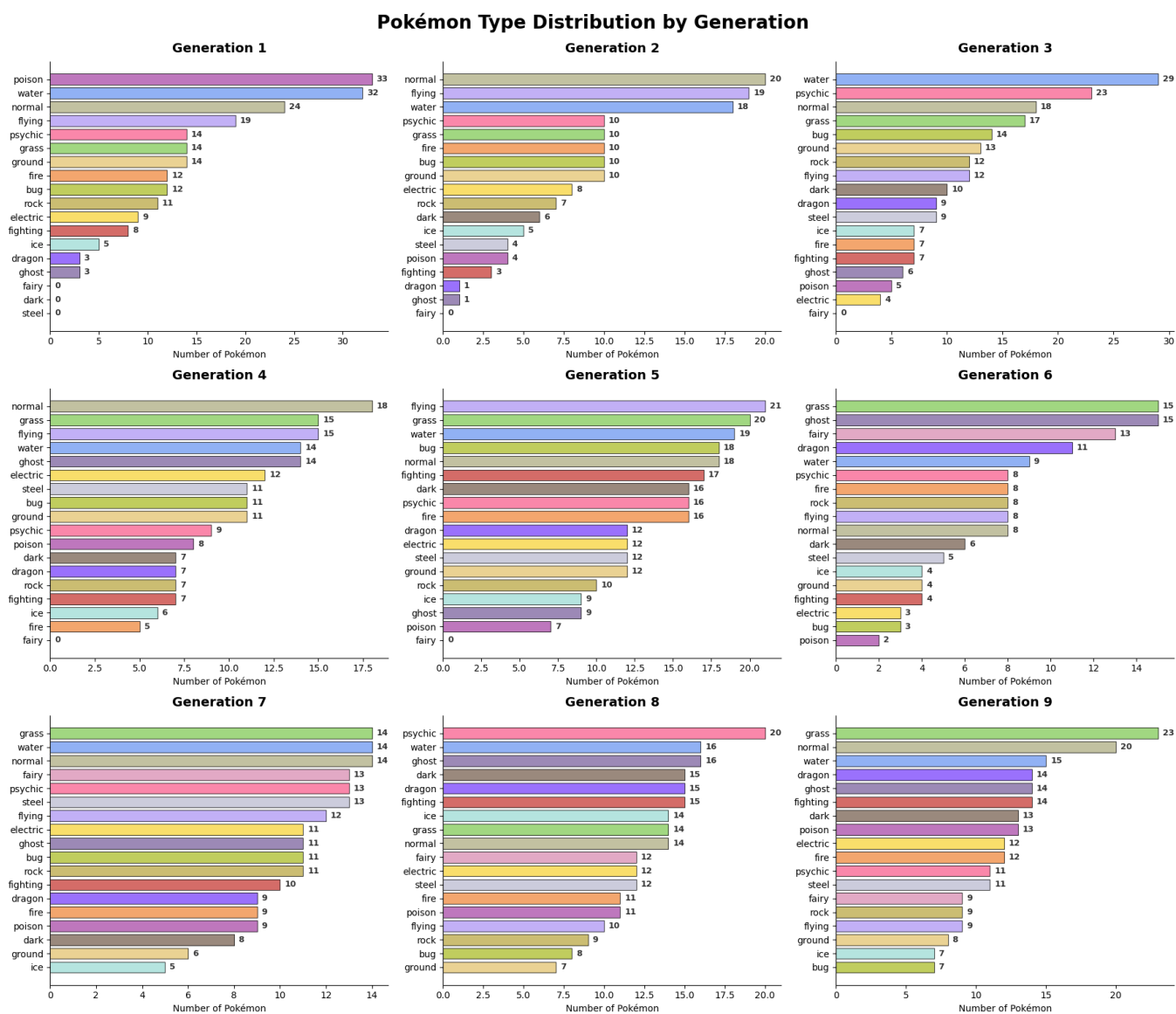


Figure 1: Type Distribution by Generation

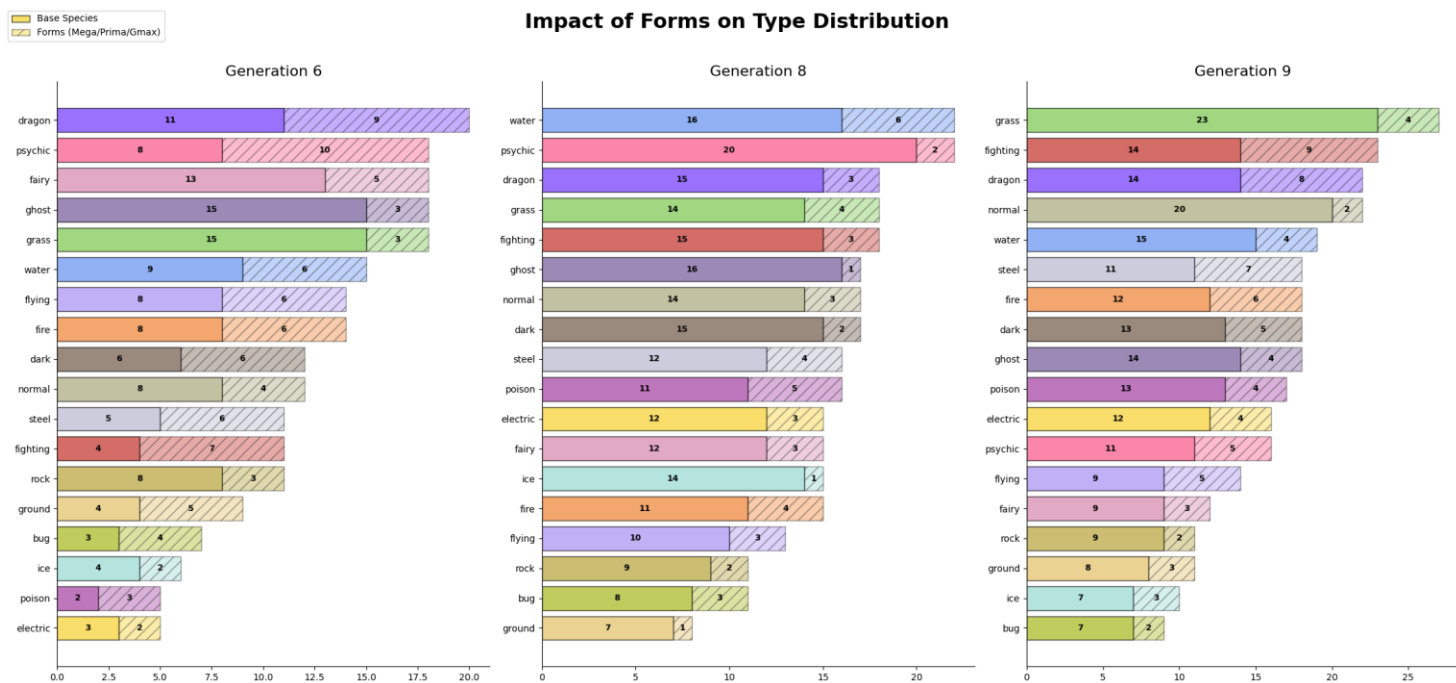


Figure 2: Impact of Forms on Type Distributions

Finally, a scatter plot was drawn to visualize the distribution of each generation within the OU (Overused) tier, the highest of the usage-based tiers. As expected, later generations were more common in OU.

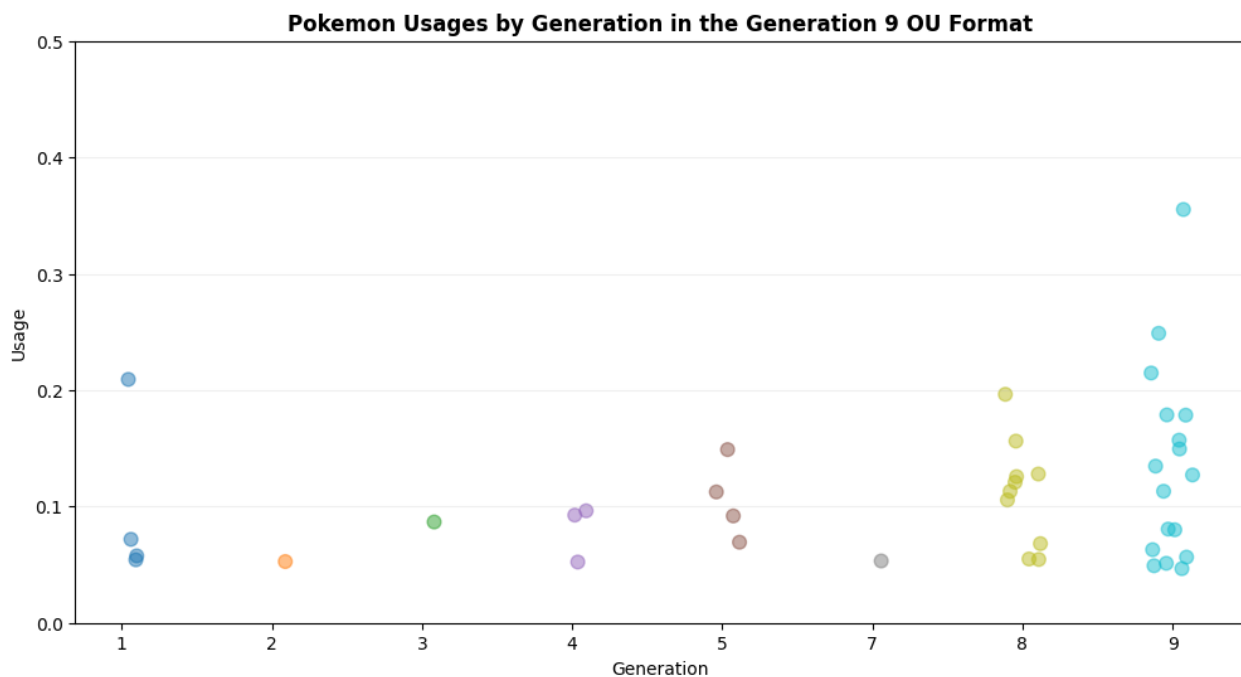


Figure 3: Usage Distribution of OU by generation

Hypothesis Testing

Three hypotheses were tested as part of this project, the first of which was: Are water types bulkier, or is Suicune gaslighting us? Historically, defensive water types like Suicune have dominated multiple generations, creating a widespread perception of the type as a defensive wall. However, I was unsure if this is an assumption built on the exceptional performance of a few or an inherent quality of the type. To test this statistically, I had to define what bulk means. True survivability relies on both typing and base stats but factoring in typing is very difficult as its highly context dependent. Dual typing introduces massive variance, a Water/Rock Pokémon has entirely different defensive liabilities than a Water/Dragon Pokémon. Given that only 84 out of the 183 Water types are mono-Water, the defensive properties of typing were not considered in this hypothesis. I quantified bulk as the proportion of a Pokémon's Base Stat Total (BST) dedicated to its defensive stats (HP, Defense, and Special Defense), using the following formula:

$$Bulk = \frac{HP + Defense + Sp.Defense}{BST}$$

Equation 1: Bulk Formula

With this metric established, I formulated my null and alternative hypothesis. The null hypothesis states that there is no statistically significant difference between the bulk distribution of Water-type Pokémon and non-Water-type Pokémon. Whereas the alternative hypothesis states Water-types have a statistically significant lead in bulk over other types. To decide on which test to apply, I plotted the bulk data as a KDE plot and a box whisker plot. The purpose of the KDE plot was to determine the shape of the distributions, to see if it was near normally distributed whereas the box plot was plotted to inspect the spread. According to the KDE plot, the bulk of both populations (water types and non-water types) were normally distributed, and the two populations are independent as such independent samples t-test was chosen. The variances of the populations were checked to pick the appropriate version of the test; variances were near equal thus the standard version of the test was used. Turns out Suicune does warp my perception. The calculated p-value was 0.3412 with a significance level of 0.05 we failed to reject the null hypothesis. Thus, it can be concluded, Water types do not have a statistically significant bulk advantage over other types.

Bulk Data Shape of Pokemon

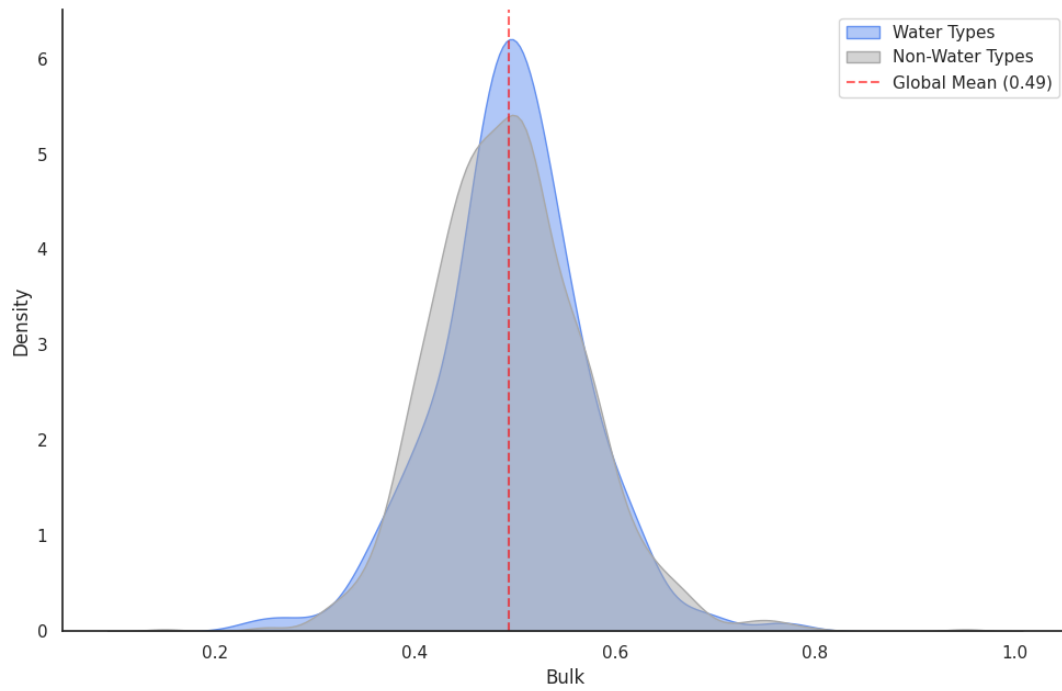


Figure 4: KDE Plot of Bulk Distributions

Bulk Whisker Box Plot

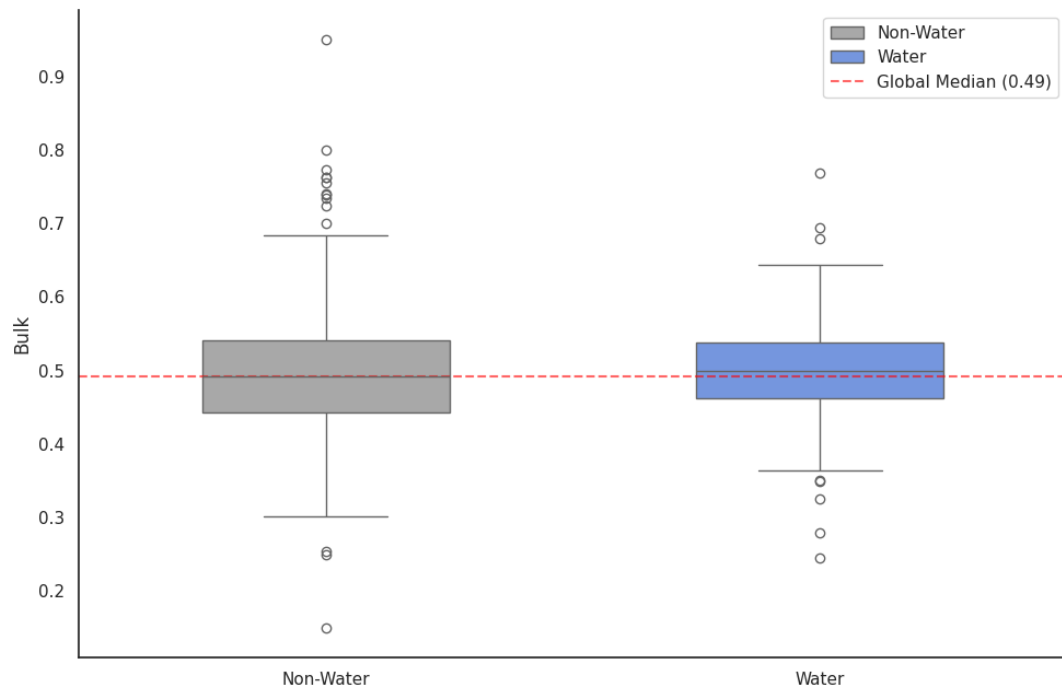


Figure 5: Box Plot of Bulk Distributions

The second hypothesis tested was directed at the impact of the physical/special split on the game design sensibilities of Pokémon. Prior to generation 4, a move's classification as physical or special depended entirely on its type for example, all Dark type moves dealt special damage, while all Fighting type moves dealt physical damage. Because of this, older Pokémon often had mediocre, balanced stats in both Attack and Special Attack just so they could effectively utilize their own typing. In generation 4 the physical/special split changed everything, removing the connection between a move's damage class and its type. I suspected that this mechanical change fundamentally altered how Pokémon were designed, allowing developers to create specialized attackers rather than jacks-of-all-trades. To measure this shift toward specialization, I quantified the impact of the physical/special split by defining a metric called the offensive gap. This metric calculates the absolute difference between a Pokémon's two attacking stats:

$$\text{Offensive Gap} = |\text{Attack} - \text{Special Attack}|$$

Equation 2: Offensive Gap Formula

A larger gap indicates a highly specialized attacker, whereas a smaller gap points to a mixed attacker. With this metric, I set up a hypothesis test. The null hypothesis states that there is no statistically significant difference between the offensive gap distributions of pre- and post-generation 4 Pokémon. The alternative hypothesis states that pre-generation 4 Pokémon have a significantly lower offensive gap. Again, a KDE plot and box plot were drawn for the same purposes. According to the KDE plot the offensive gap distributions were right skewed, with a bigger skew for the pre-generation 4 distribution. The box plot visually supported the alternative hypothesis as the median of pre-generation 4 was lower than the global median while the median of post-generation 4 was higher than the global median. Again, an independent samples t-test was applied however, due to the right skew of the distributions a non-parametric test was also performed to verify the results from the t-test. This time there was a significant difference in variances hence why Welch's t-test was used. The calculated p-value was 0.000000000008 with a significance level of 0.05 we safely rejected the null hypothesis. According to Welch's t-test there is significant evidence to back up my intuition. To verify these results Mann-Whitney U test was chosen as it is analogous to t-test. The calculated p-value was 0.000000003544 with a significance level of 0.05, the null hypothesis was rejected again. The Mann-Whitney U test supported the results from Welch's t-test thus it can be

concluded pre-gen 4 Pokémon have a lower offensive gap. These results prove the physical/special split as a turning point in Pokémon design paradigms.

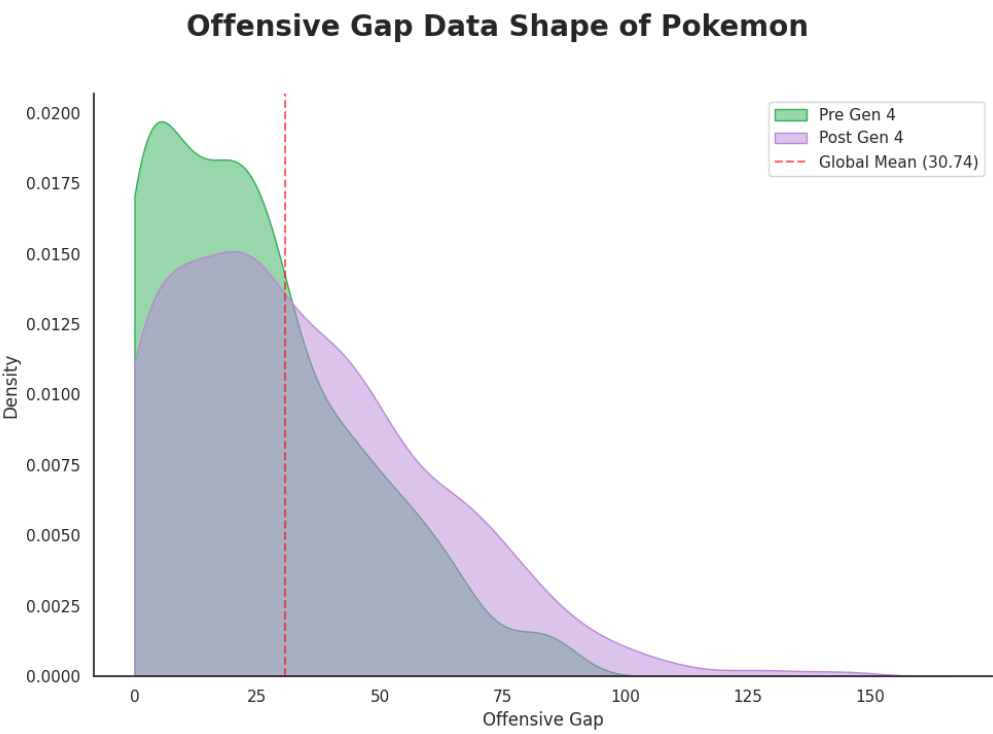


Figure 6: KDE Plot of Offensive Gap Distributions

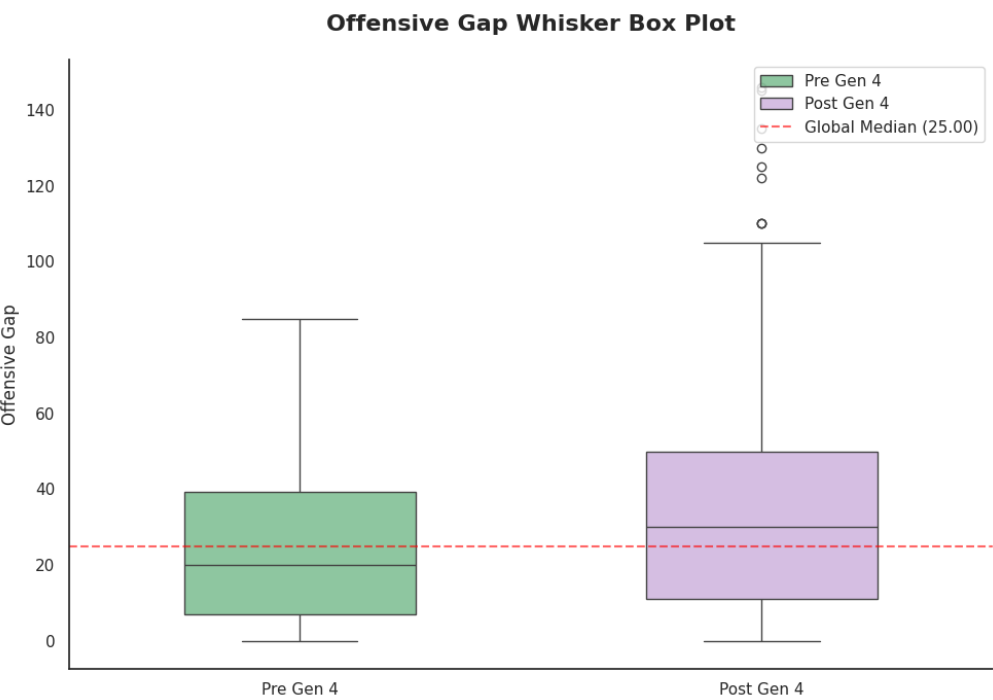


Figure 7: Box Plot of Offensive Gap Distributions

The last hypothesis examined powercreep in generation 9. Powercreep is a phenomenon where older options (in this case Pokémon) are significantly outclassed by newly released options and made obsolete. As of the time of this project, the latest generation of Pokémon is generation 9 which introduced a lot of very powerful Pokémon that dominate the competitive scene such as Great Tusk, King Gambit and Chien Pao. I have not played the generation 9 games yet, which is why I have only been exposed to the competitively well performing Pokémon of this generation through the internet. This created a perception of powercreep that may or may not be true depending on the rest of the Pokémon from this generation. As such I wanted to see if data agreed with my perception. There are 4 primary factors that make a strong Pokémon: typing, base stats, movepool and abilities. As discussed in the first hypothesis, the value of typing is context dependent. Movepool is a nuanced topic because a Pokémon does not need to have a diverse movepool to be successful it just needs access to key moves that complement its already existing qualities, this is incredibly difficult to quantify. As is the case with typing and movepool the value of abilities is difficult to quantify as they have many uses ranging from utility to damage. Which leaves us with base stats, it is the most prominent indicator of powercreep. The other 3 nuanced factors lend themselves to more subtle forms of powercreep thus if my hypothesis is supported through base stats alone then we can be sure powercreep exists in generation 9. For the reasons discussed this hypothesis focused on the base stat total (BST) distribution of generation 9 and pre-generation 9 Pokémon. The null hypothesis states there is no statistically significant difference between the BST distribution of generation 9 Pokémon and previous generation Pokémon. Whereas the alternative hypothesis claims Pokémon from generation 9 have a statistically significant BST lead over Pokémon from previous generations.

$$BST = HP + Attack + Defense + Special Attack + Special Defense + Speed$$

Equation 3: Base Stat Total Formula

The KDE plot of the BST distributions indicated a bimodal shape. The first peak is likely non-evolved Pokémon it is expected for them to have a lower base stat total than their evolved counterparts. The second peak is fully evolved Pokémon as well as single stage Pokémon (they do not evolve from or evolve into any other Pokémon). The second peak is higher because in addition to fully evolved Pokémon (which have a corresponding pre-evolution in the smaller peak) there are single stage Pokémon. The second peak is the most relevant part of the data for this hypothesis as there are very few non-fully evolved Pokémon used in competitive play. My claim is visually supported as the

second peak of the generation 9 population is further to the right. The box plot also supports the alternative hypothesis as generation 9's median is significantly higher than the global median of 458 whereas previous generation's median is slightly below the global median. Interestingly the upper whisker of previous generations stretches higher than that of generation 9 indicating previous generations have a select few Pokémon with higher BSTs despite having lower BSTs overall. As the data is non-normally distributed, Mann-Whitney U test was chosen because it makes no assumptions on the distribution of the data. The calculated p-value is 0.0006 with a significance level of 0.05 we safely reject the null hypothesis. The results of the Mann-Whitney U test show significant evidence in favor of my claim. It can be concluded that generation 9 Pokémon have a BST lead over Pokémon of previous generations thus proving the existence of powercreep.

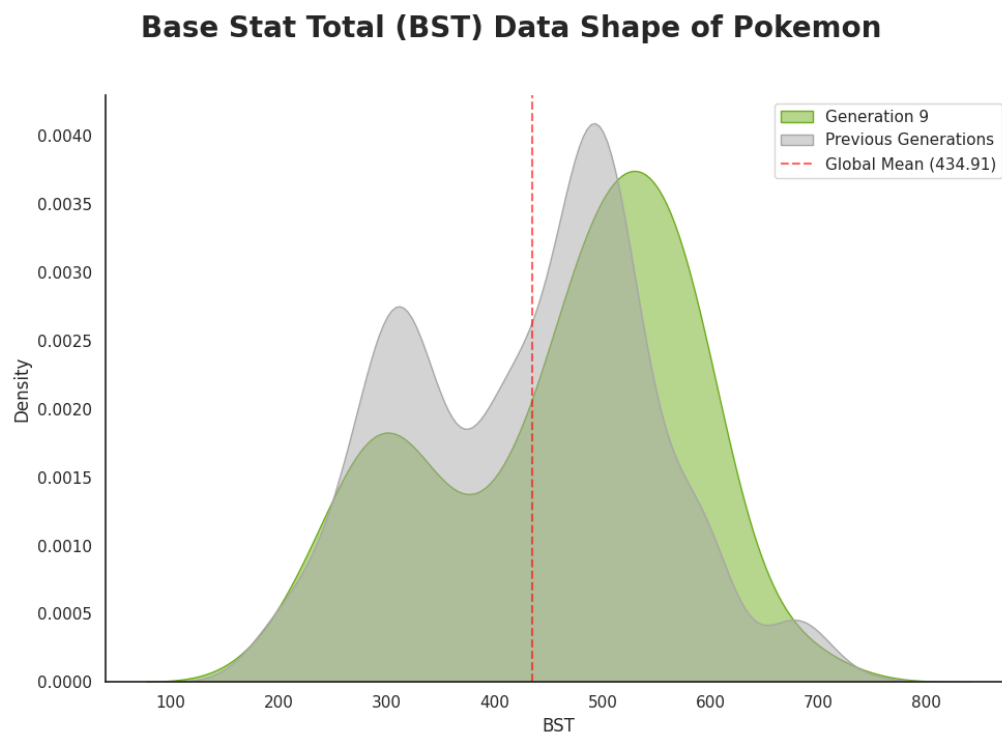


Figure 8: KDE Plot of BST Distributions

Machine Learning

My goal during ML phase of the project was to teach an ML algorithm what makes a Pokémon competitively viable. The task was framed as a 7-way classification task where a Pokémon is fed into the model, and the model is asked to identify which tier the Pokémon belongs to. If I had used the usage based statistics I have access to from the Smogon API as features it would lead to data

leakage. A very simple example is the Smogon API provides a list of Pokémon that counters a given Pokémon based on how many knockouts (KOs) or switch-outs they have achieved against the Pokémon in question. Based on this information, the model can easily infer if a Pokémon's counter is in its training set, then this Pokémon must also be in the same tier. To avoid data leakage only the inherent qualities of a Pokémon were considered as features alongside some potential role identification based on which qualities were valuable historically. In practice this way of approaching the task is unrealistic because competitive viability is highly dependent on context of the tier. For example, in generation 2 PU Furret was a top Pokémon in the format and in response Magcargo gained enough usage to be considered PU due to its favorable matchup against Furret. Magcargo was otherwise unremarkable which is why when Furret was banned from generation 2 PU it lost its single use case and dropped back to lower tiers.

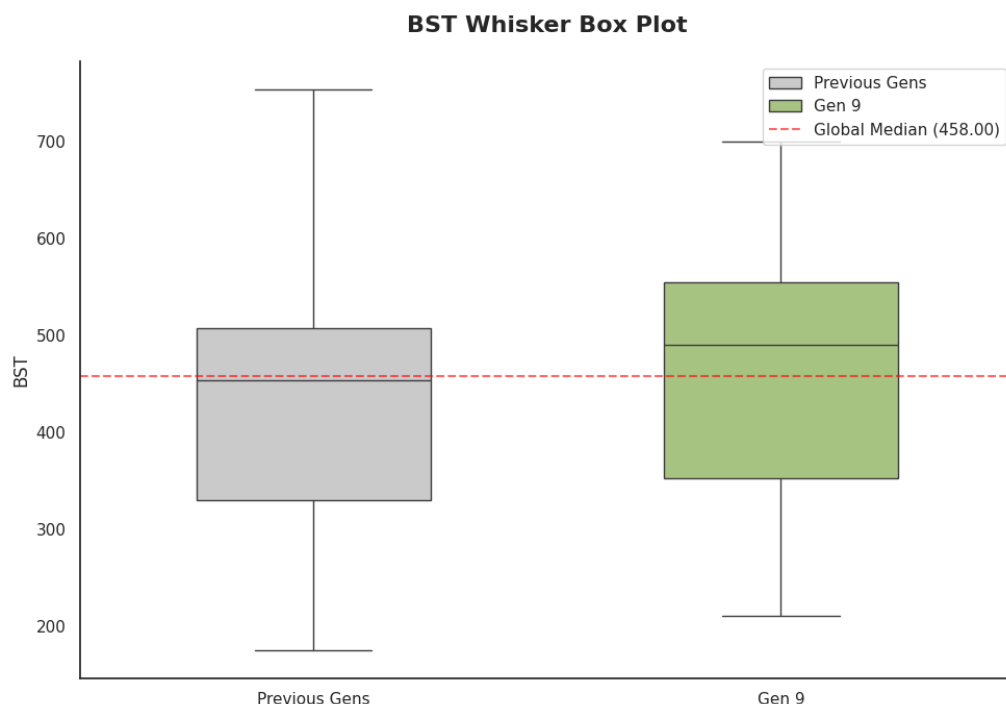


Figure 9: Box Plot of BST Distributions

The first step in the ML phase was to adapt my JSON data into a tabular format with ML ready features. I could not feed raw stats to the model otherwise it would just learn hard stat threshold cutoffs which is not a real indicator of competitive viability. For this reason, I engineered 5 features derived from base stats. Two of these (PhysBulk and SpeBulk) are a combination of the HP and the

defense or special defense stats and serve as an indication of the Pokémon's bulk. Speed is a threshold stat; it is infinitely better to have a 101 base speed compared to 100 because base 100 is a relatively common speed stat in the higher tiers. A prime example of this is Garchomp, in its base form it has 102 base speed while Mega Garchomp has base 92 despite having 40 more base attack than base Garchomp, Mega Garchomp is considered weaker for this reason. To map this way of thinking to the model a feature called the SpeedTier was created which categorizes Pokémon into 4 speed tiers. The last two base stat derived features are ratios: AtkToSpaRatio and WastedStatRatio. AtkToSpaRatio captures a Pokémon's ability to be a mixed attacker, if the ratio is close to 1 the Pokémon can run both physical and special moves, if it's lower than 1 it specializes in special attacking and if it is larger than 1, physical attacking. If a Pokémon specializes in a single damage class (physical/special) it is considered to "waste" base stats in its weaker attacking stats by the community. WastedStatRatio captures this quality by calculating how much the weaker attacking stat contributes to a Pokémon's BST.

The second type of features were based on the typing of the Pokémon. Simply multi-hot encoding all 18 types as categorical features would not have been useful. The model would associate the type combination of a strong Pokémon with the type combination being strong without knowing if that Pokémon's strength was from its other qualities. Instead, I defined two metrics called the TypeOffensiveScore and TypeDefensiveScore, these were calculated based on the number of resistances, immunities and super effective hits allowed by the given type combination (for exact formulas refer to `data_prep_and_feature_engineering.ipynb`).

Next, I wanted to give the model the ability to derive what roles a Pokémon might play based on its inherent qualities. Through my domain knowledge I created a list of moves commonly used by Pokémon in 6 different roles and multi-hot encoded these roles based on the Pokémon's movepool. The role categories were as follows: Hazard setter, Pivot, Setup, Recovery, Utility and Priority.

Finally, I needed to quantify the contribution of abilities into a Pokémon's viability. It was unrealistic to quantify the value of each ability, instead I opted to follow my approach to roles. I created a list of abilities that are incredibly useful and are not restricted to 1 or 2 Pokémon, I then multi-hot encoded these as categorical features. To avoid data leakage I created these features dynamically during k-

fold validation, a feature for an ability in my list is only created if at least one Pokémon in the current training fold has the ability.

I mapped the seven tiers ordinally from 0 to 6 and separated a 20% holdout set, making sure to stratify the split. Given how heavily populated the lower tiers are compared to the higher tiers, stratification was a must. To get a baseline performance I used Scikit-Learn's DummyClassifier using most frequent as the guessing strategy under a 5-fold stratified cross-validation setup. The baseline accuracy was 41.67% as 41.72% of the dataset is ranked ZU (the lowest tier) but its F1-score was a miserable 0.0840.

The first ML algorithm I trained was XGBoost, my reasoning for this choice was tree-based algorithms perform well with a mix of categorical and numerical features and that I desired explainability. XGBoost offered a feature importance report, given my goal was to see if a ML algorithm could learn competitive viability, I was eager to see what it valued. For hyperparameter tuning I turned to Optuna, an open-source hyperparameter optimization framework that automates hyperparameter search. I defined an Optuna study to tune the following hyperparameters: tree depth, learning rate, number of estimators, subsample ratios, column sample by tree, child weights and gamma. Goal was to maximize the macro F1-score as accuracy was a misleading performance metric due to the class imbalance. Over 35 iterations, Optuna tried various combinations of these hyperparameters within the ranges I defined, with the best trial settling on a cross-validated macro F1-score of 0.3015. I then trained an XGBoost classifier on the combined training and validation sets using the best parameters Optuna found and evaluated its performance on the holdout set. The best XGBoost model achieved a macro F1-score of 0.2782, a massive upgrade over the baseline but it was still very far from good. I plotted the confusion matrix to see where the model was struggling and discovered it was great at correctly identifying ZU Pokémon, at cost of a lot of false positives from other tiers. It also had issues distinguishing between Ubers and OU (the two highest tiers). The model really struggled in identifying the middling tiers (UU, RU, NU and PU).

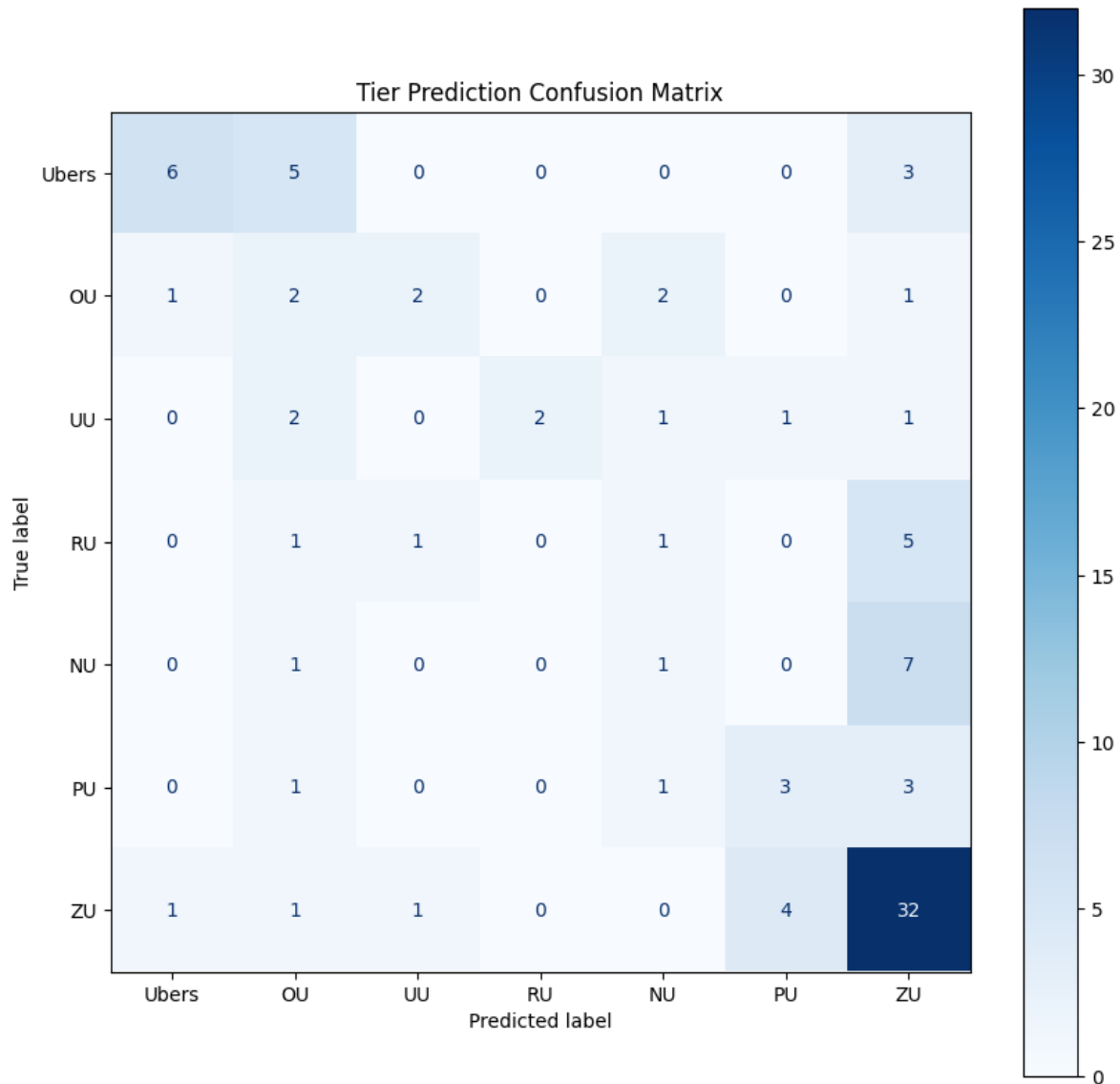


Figure 10: XGBoost Confusion Matrix

I wondered if another algorithm could handle the problem differently, so I tried a Support Vector Machine (SVM). SVMs are great at finding boundaries in multi-dimensional data, but their main draw for me was the `class_weight="balanced"` parameter, letting the model natively compensate for the tier imbalances in the dataset. Due to the distance calculations inherent to SVMs I used Scikit-Learn's standard scaler to ensure large valued features such as PhysBulk do not drown out the impact of other features. I set up another Optuna study to find the best values for the following

hyperparameters: C, gamma and hyperplane type. To my disappointment, SVM performed very similarly to XGBoost. At trial 14, the SVM optimization study hit a peak macro F1-score of 0.3096 using a polynomial kernel. Using the found best hyperparameters I trained another SVM on the combined training and validation sets then evaluated its performance on the holdout set. The best SVM model achieved a macro F1-score of 0.2704, nearly the same performance as XGBoost. The confusion matrix was very similar as well, the main difference was that SVM distributed its less confident predictions across multiple tiers, this led to significantly lower accuracy.

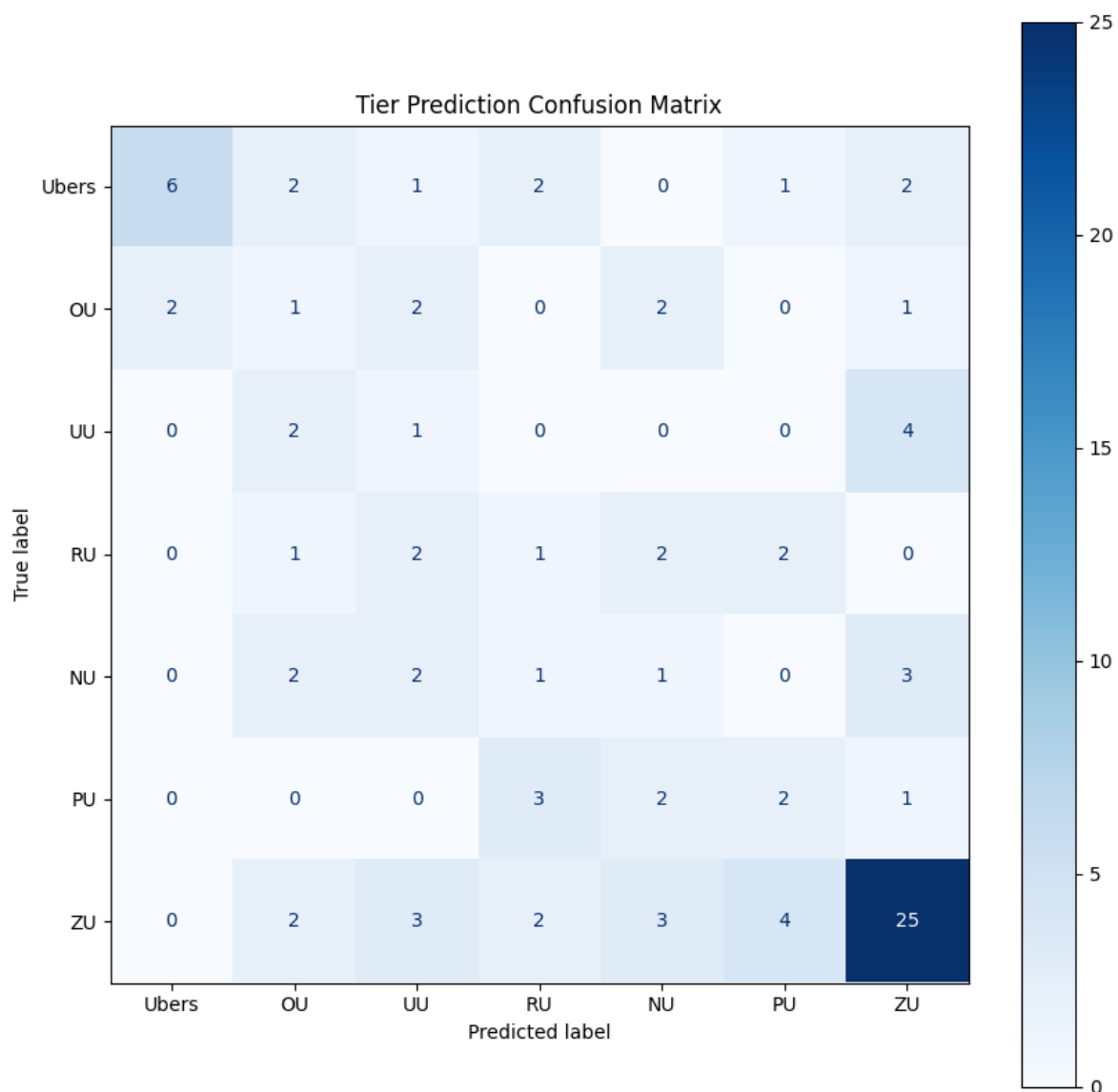


Figure 11: SVM Confusion Matrix

Findings

The key finding of my analysis is that it is extremely difficult to predict how competitively viable a Pokémon will be based on its inherent qualities. The ML attempts prove the context dependent nature of the task which is backed up by my personal experience with games. Often it takes years for players to discover what is strong and META and when they look back on what they thought was strong when the game first released, they notice that they did not understand how the game worked back then.

Limitations and future work

My current models look at each Pokémon as an isolated row in a spreadsheet. Since a Pokémon's viability is defined by its relationships with the rest of the META. The next step would be to develop methods to find ways to integrate this context without introducing data leakage. Reframing the task could be helpful in accomplishing this. Instead of asking a model, "Is this Pokémon strong?" I could ask, "How does this Pokémon disrupt the existing dominant threats?" This would shift the problem from a classification task to an ecosystem simulation, which more accurately represents how a community solves a metagame.

Instead of feature engineering an embedding approach could be considered to capture more subtle relationships that make certain tools more powerful when certain Pokémon have access to them. For example, pivot moves are often more powerful when used by fast Pokémon as it allows them to deal damage while avoiding retaliation.

In this study I focused on the current snapshot of the generation 9 metagames, but Pokémon is a 30 year old franchise. Instead of focusing on a snapshot a temporal approach could be used to teach a model the evolution of different metagames throughout the franchises lifespan in hopes of identifying trends within the progression of metagames.

References

- Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework [Conference paper]. Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining, 2623–2631. <https://doi.org/10.1145/3292500.3330701>
- Chen T, He T, Benesty M, Khotilovich V, Tang Y, Cho H, Chen K, Mitchell R, Cano I, Zhou T, Li M, Xie J, Lin M, Geng Y, Li Y, Yuan J, Cortes D (2026). *xgboost: Extreme Gradient Boosting*. R package version 3.3.0.0, <https://github.com/dmlc/xgboost>.
- Hallett, P. (2014, December 4). *PokéAPI*. Pokéapi. <https://pokeapi.co/>
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020, September 16). *Array programming with NumPy*. Nature News. <https://www.nature.com/articles/s41586-020-2649-2>
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- Mulla, R. (2025, September 10). *Fast tuning with xgboost 3.0 + optuna + gpus*. Kaggle. <https://www.kaggle.com/code/robikscube/fast-tuning-with-xgboost-3-0-optuna-gpus#Key-XGBoost-Parameters>
- Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, CJ Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E.A. Quintero, Charles R Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. (2020) SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17(3), 261–272. DOI: [10.1038/s41592-019-0686-2](https://doi.org/10.1038/s41592-019-0686-2).
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. <https://jmlr.csail.mit.edu/papers/v12/pedregosa11a.html>
- Sampaio, C. (2023, July 2). *Understanding SVM hyperparameters*. Stack Abuse. <https://stackabuse.com/understanding-svm-hyperparameters/>
- Scheibelhut, K. (2019, August 22). *PKMN/smogon: Wrapper around Smogon's analyses and usage statistics*. GitHub. <https://github.com/pkmn/smogon>
- Shrestha, P. (2024, September 9). *Tuning SVM hyperparameters: Making your Classifier Shine like a pro!*. Medium. <https://medium.com/@prayushshrestha89/tuning-svm-hyperparameters-making-your-classifier-shine-like-a-pro-8673639ddb16>

Smogon Contributors. (2004). *Smogon*. Smogon University - Competitive Pokémon Community.
<https://www.smogon.com/>

The pandas development team. pandas-dev/pandas: Pandas [Computer software].
<https://doi.org/10.5281/zenodo.3509134>

Waskom, M. L., (2021). seaborn: statistical data visualization. Journal of Open Source Software, 6(60), 3021, <https://doi.org/10.21105/joss.03021>.

Note: For LLM assistance refer to the /llm_use_documentation folder of the repo